

Stage 1.4.2.5.4 — Docling Table / Image / Formula Understanding The objective is to answer:

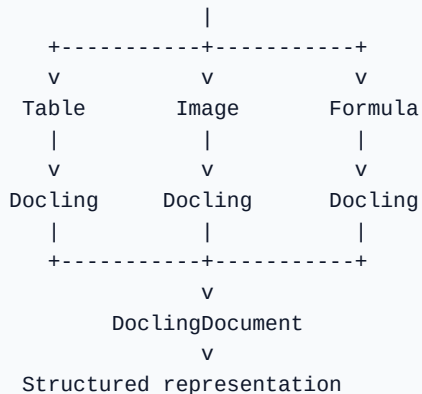
PIPELINE FLOW

When Docling receives a complex PDF, how does it recognize and represent tables, images/figures, and formulas?

Our flow will be:

PIPELINE FLOW

PDF



I've also created a dedicated test PDF for this stage.

Download the Stage 1.4.2.5.4 sample PDF

It contains:

- a structured table
- an architecture diagram as an embedded image
- a mathematical formula
- captions and explanatory text

Step 1 — Verify your Docling version

You already told me you're using Docling 2.120.3, but let's verify directly in the notebook.

PYTHON

```
import docling
```

PYTHON

```
print("Docling version:", docling.__version__)
```

Expected:

OUTPUT

```
Docling version: 2.120.3
```

Step 2 — Define the sample PDF

Use the PDF we just generated.

PYTHON

```
from pathlib import Path
```

```
pdf_path = Path(
    "stage_1_4_2_5_4/docling_table_image_formula_sample.pdf"
)
```

PYTHON

```
print(pdf_path)
print(pdf_path.exists())
You should get:
```

STRUCTURE / CODE

```
stage_1_4_2_5_4/docling_table_image_formula_sample.pdf
```

```
True
```

If you place the PDF somewhere else in your RAG project's data/raw/pdf/ directory, simply change the path.

Step 3 — Import Docling

PYTHON

```
from docling.document_converter import DocumentConverter
Create the converter:
```

PYTHON

```
converter = DocumentConverter()
```

Step 4 — Convert the PDF

PYTHON

```
result = converter.convert(pdf_path)
```

STRUCTURE / CODE

```
doc = result.document
```

Now we have:

PIPELINE FLOW

```
PDF
  v
DocumentConverter
  v
ConversionResult
  v
DoclingDocument
Remember this distinction:
```

STRUCTURE / CODE

```
result
is the conversion result.
```

result.document

is the actual:

DoclingDocument

Step 5 — First inspect the Markdown

We deliberately do this, but we will not rely on it as our only inspection method.

STRUCTURE / CODE

```
markdown_text = doc.export_to_markdown()
```

PYTHON

```
print(markdown_text)
You should see the ordinary text and table represented in Markdown.
For example, the table should look approximately like:
```

STRUCTURE / CODE

```
| Stage | Component | Purpose |
|---|---|---|
| 1 | Event Producer | Publishes business events |
| 2 | Azure Event Hubs | Ingests and partitions event streams |
...
```

The picture may appear as a placeholder depending on the export settings.
 Docling's `export_to_markdown()` supports image modes such as `placeholder`, `embedded`, and `referenced` images. (Docling Project)

Step 6 — Inspect the tables directly [*]

This is where our learning becomes more interesting.

Run:

PYTHON

```
print("Number of tables:", len(doc.tables))
Then:
```

PYTHON

```
for i, table in enumerate(doc.tables):
    print(f"\n===== TABLE {i + 1} =====")
    print(table.export_to_markdown(doc))
We are no longer asking:
```

STRUCTURE / CODE

"What did Markdown export give me?"
We're asking:

STRUCTURE / CODE

"What table objects did Docling actually create?"
Docling exposes table items and supports exporting an individual table to Markdown, HTML, and other structured representations. (Docling Project)

Step 7 — Export the table as a DataFrame

This is extremely useful for RAG later.

PYTHON

```
for i, table in enumerate(doc.tables):
    print(f"\n===== TABLE {i + 1} =====")
```

```
df = table.export_to_dataframe(doc)
```

```
display(df)
```

Conceptually:

PIPELINE FLOW

PDF Table

↓

Docling TableItem

↓

DataFrame

You should get something similar to:

```
StageComponentPurpose1Event ProducerPublishes business events2Azure Event HubsIngests and partitions
event streams3Stream ConsumerReads and transforms events4Azure Data ExplorerStores data for analyti
cs
```

This is much more powerful than simply extracting text.

Step 8 — Understand the table structure

Now inspect the first table:

STRUCTURE / CODE

```
table = doc.tables[0]
```

PYTHON

```
print(type(table))
```

Then:

PYTHON

```
print(table)
```

And inspect its available methods:

```
[m for m in dir(table) if not m.startswith("_")]
```

Pay particular attention to methods related to:

STRUCTURE / CODE

```
export
cells
captions
image
provenance
```

The important concept is:

DOCUMENT STRUCTURE

```
TableItem
|
+-- rows
+-- columns
+-- cells
+-- caption
+-- provenance
+-- structure
```

This is why table-aware ingestion is different from ordinary text extraction.

Step 9 — Now investigate images / pictures

Run:

PYTHON

```
print("Number of pictures:", len(doc.pictures))
```

Then:

PYTHON

```
for i, picture in enumerate(doc.pictures):
    print(f"\n===== PICTURE {i + 1} =====")
    print(type(picture))
    print(picture)
```

This tells us whether Docling detected our architecture diagram as a `PictureItem`. Docling provides `PictureItem` support and can retrieve the corresponding image from the `DoclingDocument`. (Docling Project)

Step 10 — Extract the detected image

If we have at least one picture:

STRUCTURE / CODE

```
if len(doc.pictures) > 0:
```

```
    picture = doc.pictures[0]
```

STRUCTURE / CODE

```
    picture_image = picture.get_image(doc)
```

PYTHON

```
    print(type(picture_image))
    If the image is available, save it:
```

if picture_image is not None:

STRUCTURE / CODE

```
    picture_image.save(
        "docling_extracted_picture.png"
    )
```

PYTHON

```
    print("Picture saved.")
    Now you've demonstrated:
```

PIPELINE FLOW

```
PDF
  v
Embedded architecture diagram
  v
Docling
  v
PictureItem
  v
PIL Image
  v
PNG
This is very different from OCR.
```

Step 11 — Important distinction: Image Detection vs Image Understanding

This is one of the most important concepts in this stage.

Suppose Docling tells us:

PictureItem

That means:

Docling identified a picture/figure region.

It does not necessarily mean:

STRUCTURE / CODE

"Docling understands that this is an Azure Event Hubs -> Stream Consumer -> Data Explorer architecture."
Those are two different levels:

STRUCTURE / CODE

Level 1
Image detection
v
"There is a picture here."
versus:

STRUCTURE / CODE

Level 2
Image understanding
v
"This is an event-processing architecture."
For the second task, vision-language capabilities can be involved. Docling supports picture classification and picture-description options in its pipeline configuration. (Docling Project)
That distinction will become extremely important when we reach your multimodal RAG stage.

Step 12 — Investigate the formula

Our sample PDF contains:

STRUCTURE / CODE

$P(A|B) = P(B|A)P(A) / P(B)$
The formula is deliberately included as an image for this first experiment.
Let's inspect the document's text items.

for item, level in doc.iterate_items():

PYTHON

```
print(
    type(item).__name__,
    getattr(item, "label", None),
    getattr(item, "text", "")
)
```

Look for anything whose label indicates:

STRUCTURE / CODE

FORMULA
The exact result is something we want to observe, not assume.

Step 13 — Understand formula enrichment

PIPELINE FLOW

This is where Docling becomes more interesting.

Docling's PDF pipeline has a `do_formula_enrichment` option for mathematical formula recognition and conversion to LaTeX. (Docling Project)

Conceptually:

PDF

↓

Formula region

↓

Formula recognition

↓

LaTeX

For example, ideally:

STRUCTURE / CODE

$P(A|B) = P(B|A)P(A) / P(B)$
could become something similar to:

STRUCTURE / CODE

$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$

However, we should not turn this option on blindly in your Docling 2.120.3 environment.

Your installed version is important, and formula enrichment can require additional model dependencies/resources. The current Docling documentation confirms that formula enrichment is a specialized processing option and that enabling multiple enrichment features increases processing time. (Docling Project)

So our first experiment is intentionally:

Detect and inspect first -> enable specialized enrichment only after we understand the baseline.

Step 14 — Inspect all document elements together

Now let's create one useful diagnostic cell:

for item, level in doc.iterate_items():

STRUCTURE / CODE

```
label = getattr(item, "label", None)
```

STRUCTURE / CODE

```
text = getattr(item, "text", "")
```


PYTHON

```
print(
    f"Type={type(item).__name__:<20} "
    f"Label={str(label):<20} "
    f"Text={text[:100]!r}"
)
```

This is one of the most important cells in this stage.
We are beginning to see:

PYTHON

```
DoclingDocument
```

```
|
```

```
+-- TextItem
```

```
+-- SectionHeaderItem
```

```
+-- TableItem
```

```
+-- PictureItem
```

```
+-- ...
```

```
+-- Formula-related element, if detected
```

Now we're actually learning the Docling document model, rather than treating Docling as a black-box PDF-to-Markdown converter.

Step 15 — Export the complete structured document

Since you previously encountered:

PYTHON

```
AttributeError:
```

```
DoclingDocument has no attribute export_to_json
```

```
we must remember the correct API for your Docling v2 workflow:
```

STRUCTURE / CODE

```
document_dict = doc.export_to_dict()
```

Docling v2 moved document export operations onto DoclingDocument; the official v2 documentation explicitly shows `export_to_dict()`, `export_to_markdown()`, and `export_to_document_tokens()`. (GitHub)

You can inspect:

STRUCTURE / CODE

```
document_dict.keys()
```

and save it:

PYTHON

```
import json
```

PYTHON

```
output_json = Path(
    "docling_table_image_formula.json"
)
```

PYTHON

```
output_json.write_text(
    json.dumps(
        document_dict,
        indent=2,
        ensure_ascii=False,
    ),
    encoding="utf-8",
)
```

PYTHON

```
print(output_json)
This JSON becomes very useful for debugging.
```

Step 16 — Our first Table / Image / Formula summary

After running the notebook, create this summary:

PYTHON

```
print("==== DOCLING CONTENT SUMMARY =====")
```

PYTHON

```
print("Tables   :", len(doc.tables))
print("Pictures :", len(doc.pictures))
```

STRUCTURE / CODE

```
formula_items = []
```

for item, level in doc.iterate_items():

STRUCTURE / CODE

```
label = str(getattr(item, "label", ""))
```

```
if "formula" in label.lower():
    formula_items.append(item)
```

PYTHON

```
print("Formulas :", len(formula_items))
Ideally we'll get something like:
```

STRUCTURE / CODE

```
==== DOCLING CONTENT SUMMARY =====
```

Tables : 1

Pictures : 2

Formulas : ...

Don't worry if the formula count isn't what we expect yet. That itself is part of the experiment: our formula is

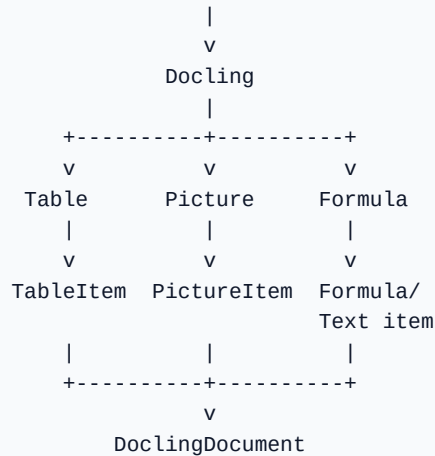
intentionally an image, so the baseline pipeline may treat it as a picture rather than as a semantic formula.

Step 17 — What we've learned from this experiment

The most important mental model is:

PIPELINE FLOW

PDF



And each content type needs a different downstream strategy.
ContentWhat we want from DoclingRAG implicationTextText + hierarchyNormal text chunkTableRows/columns/cellsStructure-aware chunkImagePicture + locationVision processingFormulaFormula/LaTeX representat
ionFormula-aware textCaptionAssociation with elementPreserve with elementOne important correction fr
om our previous stage
This stage also helps explain the Analytics Layer problem you found.
We should never assume:

Looks like a table

v

Must be TableItem

or:

Looks like two columns

v

Must be two independent text regions

The PDF's internal structure matters.

That's exactly why we're now explicitly inspecting:

doc.tables
doc.pictures
doc.iterate_items()
doc.export_to_dict()
rather than only:
doc.export_to_markdown()

Where we are after this implementation

Our Docling track now becomes:

PIPELINE FLOW

Stage 1.4.2.5

OCR Approach

```

|
+-- 1.4.2.5.1 – Basic OCR
|   [DONE]
|
+-- 1.4.2.5.2 – Docling for Scanned PDFs
|   [DONE]
|
+-- 1.4.2.5.3 – Docling Layout & Reading Order
|   [DONE]
|
+-- 1.4.2.5.4 – Table / Image / Formula Understanding
|   [CURRENT] CURRENT
|
+-- 1.4.2.5.5 – Docling -> LangChain -> RAG
    [PENDING]

```

Do not move to 1.4.2.5.5 yet.

First, run the notebook cells above against the new PDF. The important outputs for our next step will be:

STRUCTURE / CODE

1. `len(doc.tables)`
2. `len(doc.pictures)`
3. `table.export_to_markdown(doc)`
4. `table.export_to_dataframe(doc)`
5. `picture.get_image(doc)`
6. output of `doc.iterate_items()`
7. formula-related items, if any
8. `doc.export_to_markdown()`

That will let us investigate exactly how your Docling 2.120.3 installation represents each content type, rather than relying on assumptions from newer Docling documentation.